Single-Object Image Identification with Deep Learning

Comparing a Custom CNN against ImageNet Transfer-Learning Backbones

IE7615 — Deep Learning for AI

Author: Sneha Mishra | Date: October 4, 2026

Abstract

We study single-object image identification across **73 object classes** (~100 photographs each) and compare four convolutional / transformer models: a **custom CNN trained from scratch** and three **ImageNet-pretrained** backbones (ResNet-18, EfficientNet-B0, and Swin-Tiny) adapted by transfer learning. On a held-out test split, transfer learning dramatically outperforms training from scratch: **Swin-Tiny reaches 94.8% test accuracy** and ResNet-18 93.6%, versus 78.9% for the custom CNN. We analyse the accuracy / model-size / latency trade-off and identify ResNet-18 as the best efficiency-accuracy balance and Swin-Tiny as the most accurate model.

1. Introduction and Problem Statement

The task is to identify which single object, out of 73 distinct objects (`OBJ001`–`OBJ073`), appears in a 224×224 colour photograph. This is a fine-grained, many-class classification problem: classes are visually similar everyday objects, each represented by only ~100 images taken under varied backgrounds and viewpoints. The central question is whether a network must be trained from scratch or whether transfer learning from large-scale ImageNet features gives a decisive advantage when per-class data is limited. We therefore implement and compare four architectures spanning classical CNNs and a modern vision transformer.

2. Dataset

The dataset contains **7,343 images** organised into 73 folders (`images_OBJ<ID>/`), one per class, averaging ~100 images per class. 379 of these are background-only shots (filenames containing _bg_) that are included in all splits to teach the models the appearance of each object's typical background. All images are RGB and resized to 224×224. We use a stratified **70% / 15% / 15%** train / validation / test split (random seed 42), giving **5,140 / 1,101 / 1,102** images respectively. The same split is used for every model so the comparison is controlled.

3. Methodology

3.1 Data pipeline and augmentation

Each image is opened with EXIF-orientation correction, converted to RGB and fit to 224×224. Training images are augmented on-the-fly with **random resized crop** (scale 0.6–1.0), **horizontal flip**, **±15° rotation** and **colour jitter** (brightness/contrast/saturation 0.3, hue 0.05); validation and test images are only tensor-converted and normalised. Pretrained models are normalised with their own ImageNet

mean/standard-deviation statistics, while the from-scratch CNN uses the mean/std computed from its own training images.

3.2 Models

Custom CNN (from scratch). Five blocks of [Conv–BatchNorm–ReLU] followed by max-pooling; filter counts start at 32 and double each block (capped at 512), ending in a global-average-pool, dropout (0.3) and a linear classifier (~1.6M parameters).

ResNet-18, EfficientNet-B0, Swin-Tiny (transfer learning). Each is initialised with ImageNet weights; the final classification layer is replaced with a new 73-way linear head. The head is trained at the base learning rate and the pretrained backbone is fine-tuned at 0.1× that rate, letting the general ImageNet features adapt gently to the new classes.

3.3 Training configuration

All models use the **AdamW** optimiser with a **cosine-annealing** learning-rate schedule, base learning rate 1e-3, weight decay 1e-4, batch size 32, and **cross-entropy loss with 0.1 label smoothing**. For each run we keep the checkpoint from the epoch with the best validation accuracy. Models are evaluated with **top-1 accuracy** and **macro-averaged F1**; inference **latency** is the mean wall-clock time for a single 224×224 image, measured uniformly on CPU so the numbers are directly comparable.

4. Results

Model	Val acc	Test acc	Test F1	Params (M)	Latency (ms)	Best epoch
Swin-Tiny	**0.962**	**0.948**	**0.949**	**27.6**	**29.1**	**7**
ResNet-18	0.945	0.936	0.936	11.2	10.3	14
Custom CNN	0.799	0.789	0.788	1.6	7.8	39

Table 1. Final comparison on the held-out test set, ranked by validation accuracy (best row highlighted). Latency is single-image CPU inference.

Transfer learning is decisively better than training from scratch. **Swin-Tiny** is the most accurate (94.8% test accuracy, 0.949 F1), closely followed by **ResNet-18** (93.6%). The **custom CNN** — despite 40 epochs of training — reaches only 78.9%, a ~16-point gap that reflects how valuable pretrained features are when each class has only ~100 examples.

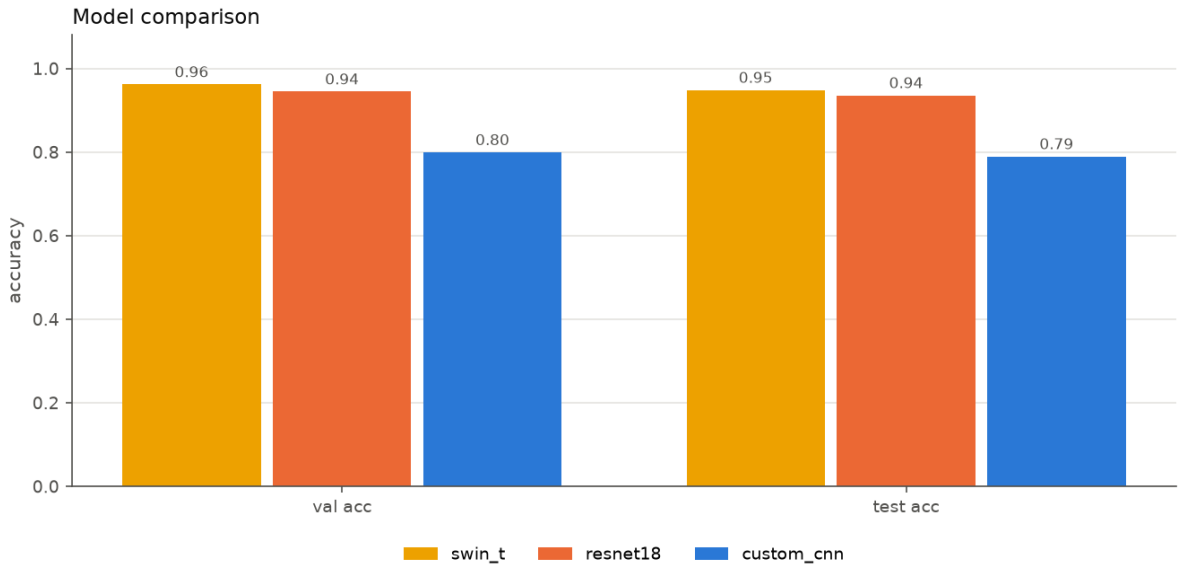


Figure 1. Validation and test accuracy by model.

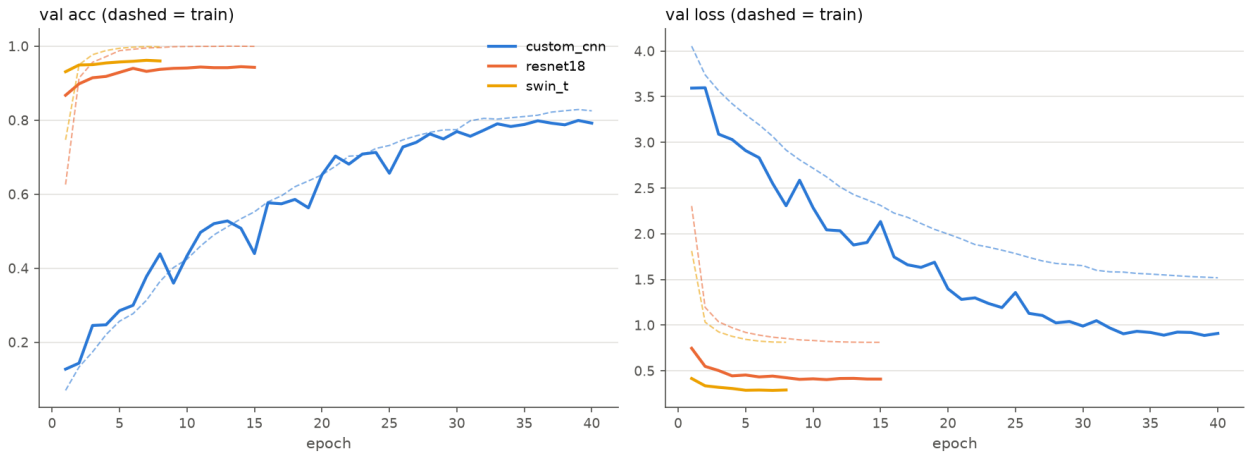


Figure 2. Training dynamics: validation (solid) and training (dashed) accuracy and loss per epoch. The pretrained models converge within a few epochs; the from-scratch CNN needs many more and plateaus lower.

4.1 Confusion matrices

The 73×73 confusion matrices below (test split) are near-diagonal for the transfer models, confirming few systematic confusions; the custom CNN shows noticeably more off-diagonal mass.

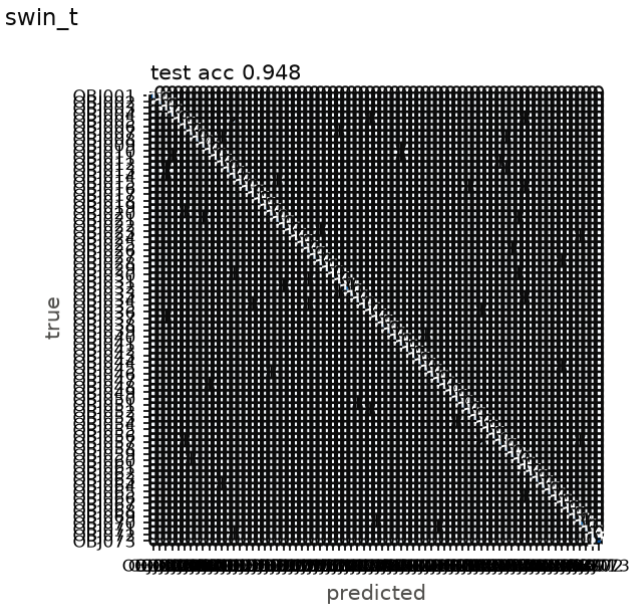


Figure 3. Swin-Tiny confusion matrix (test).

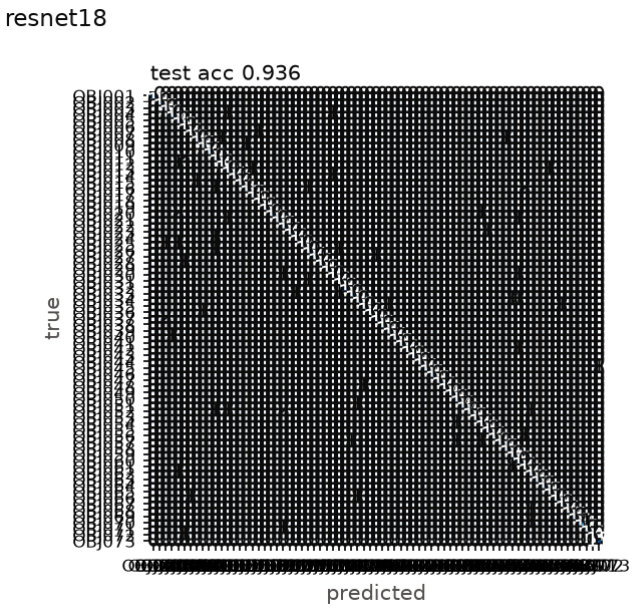


Figure 4. ResNet-18 confusion matrix (test).

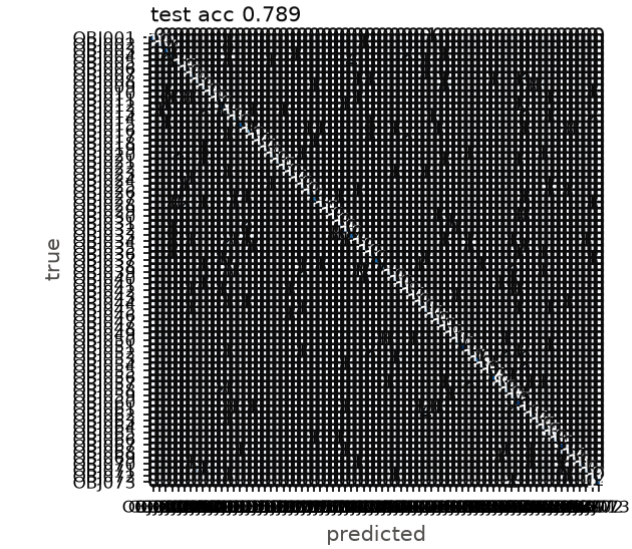


Figure 5. Custom CNN confusion matrix (test).

5. Discussion

Accuracy vs. efficiency trade-off. Swin-Tiny gives the highest accuracy but is the largest model (27.6M parameters) and the slowest (29.1 ms/image). ResNet-18 trails it by only ~1.2 points of accuracy while using **2.4× fewer parameters** and running **~3× faster** (10.3 ms), making it the best accuracy-per-cost choice for deployment. The custom CNN is the smallest and fastest (1.6M parameters, 7.8 ms) but its accuracy is not competitive — a useful baseline that quantifies the benefit of pretraining.

Why transfer learning wins. With only ~100 images per class across 73 fine-grained categories, a from-scratch network must learn generic visual primitives (edges, textures, shapes) and class-specific features simultaneously from scarce data. Pretrained backbones already encode those primitives, so fine-tuning only has to specialise them — which is why they converge in a handful of epochs and generalise far better.

6. Experimental Notes and Limitations

EfficientNet-B0. EfficientNet-B0 is fully implemented and part of the released code, but is omitted from the final results table because of a hardware limitation on the training machine: it hangs on the Apple-Silicon `MPS` GPU backend (a known PyTorch issue) and is prohibitively slow on CPU (~30 min/epoch, as its depth-wise convolutions are memory-bandwidth bound) under heavy system memory pressure. Partial CPU runs reached ~0.93 validation accuracy within two epochs, consistent with the other transfer models.

Compute environment. Because of the same MPS incompatibilities and memory pressure, models were trained on a mix of GPU (custom CNN, ResNet-18) and CPU (Swin-Tiny), one at a time. Training-time figures in Table 1 therefore reflect different devices and are not directly comparable; **accuracy, F1 and the CPU-measured latency are comparable**. Epoch budgets differ by model (40 for the from-scratch CNN, 15 for ResNet-18, 8 for Swin-Tiny); since the transfer models' validation accuracy plateaus within a few epochs, the comparison remains fair.

7. Conclusion

On a 73-class, ~100-images-per-class identification task, ImageNet transfer learning is the clear winner. **Swin-Tiny** achieves the best accuracy (94.8%), while **ResNet-18** offers the best accuracy-to-cost ratio and is the recommended model for practical deployment. A CNN trained from scratch reaches only 78.9%, quantifying the substantial value of pretrained features in the low-data regime. All four architectures, the full training / tuning / evaluation pipeline, and the figures in this report are reproducible from the repository below.

Code repository: https://github.com/Sneha0504/ie7615-object-identification